

DuocUC



Ingeniería de soluciones con Inteligencia Artificial

ISY0101-002D

San Bernardo

Softech

Felipe Monsalve

Gabriel Berman

Francisco Andres Macaya Matas

[GitHub](#)

Abril 2026

ÍNDICE DE CONTENIDO

1. Análisis del caso organizacional (Apartado A)	1
1.1 Contexto de SoftTech Solutions y el problema de soporte técnico.	1
1.2 Objetivos de la implementación de IA.	2
1.3 Requerimientos específicos y restricciones de privacidad.	3
1.4 Respuestas a Consultas de Evaluación (Requerimientos del Docente)	3
2. Arquitectura de la solución (Apartado D)	5
2.1 Diagrama de arquitectura del sistema.	5
2.2 Descripción de componentes (Orquestador, VectorDB, LLM, Interfaz).	6
3. Diseño e implementación de pipeline RAG (Apartado C)	7
3.1 Fuentes de datos internas (Manuales PDF, API, Historial de tickets).	7
3.2 Flujo de ingesta, fragmentación (Chunking) y vectorización.	7
3.3 Mecanismos de recuperación de datos.	7
4. Formulación de prompts (Apartado B)	8
4.1 Diseño del System Prompt del agente.	8
4.2 Justificación de la estructura para evitar alucinaciones y mantener el rol.	9
5. Documentación técnica (Apartado E)	9
5.1 Justificación de decisiones de diseño tecnológico	9
5.2 Evidencia de pruebas y control de alucinaciones	10
6. Conclusiones y Reflexiones Individuales	13
6.1 Conclusión General del Proyecto	13
6.2 Reflexión Personal - Felipe Monsalve	14
6.3 Reflexión Personal - Gabriel Bernal	14
7. Referencias Bibliográficas	15

1. Análisis del caso organizacional (Apartado A)

1.1 Contexto de SoftTech Solutions y el problema de soporte técnico.

SoftTech Solutions es una organización proveedora de un software en la nube (SaaS) especializado en la gestión de inventarios y logística a gran escala, la cual da servicio a una base de más de 500 clientes corporativos. El ecosistema del software es de alta complejidad y se apoya en una extensa documentación técnica que incluye manuales de usuario, guías de integración API y documentos de resolución de errores actualizados mensualmente.

El problema crítico que enfrenta la organización es el desbordamiento de su equipo de soporte técnico humano. Actualmente, el 60% de los tickets de soporte ingresados corresponden a consultas operativas recurrentes de Nivel 1 (por ejemplo, "¿Cómo configurar la alerta de stock mínimo?"), cuya información ya se encuentra documentada de manera oficial. Este volumen de consultas repetitivas genera un impacto negativo directo en la calidad del servicio, provocando tiempos de espera de hasta 24 horas para resoluciones simples y saturando al personal técnico, lo que limita su capacidad para atender incidencias de programación o arquitectura de mayor complejidad.

1.2 Objetivos de la implementación de IA.

Para mitigar esta problemática, se propone la implementación de una solución basada en Inteligencia Artificial con los siguientes objetivos:

- **Objetivo General:** Automatizar la resolución de consultas técnicas de Nivel 1 mediante la integración de un agente de IA en el flujo de atención al cliente.
- **Objetivos Específicos:**
 - Reducir el tiempo de respuesta inicial de atención al cliente desde un máximo de 24 horas a menos de 10 segundos.
 - Filtrar y resolver de manera autónoma al menos el 50% de las consultas básicas, disminuyendo la carga directa sobre los ejecutivos humanos.
 - Garantizar que el 100% de las respuestas generadas sean técnicamente precisas y fundamentadas de manera exclusiva en la documentación oficial provista por la empresa.

1.3 Requerimientos específicos y restricciones de privacidad.

El diseño del sistema debe someterse a estrictas reglas de negocio y restricciones técnicas para su viabilidad corporativa:

- **Precisión Crítica (Mitigación de Alucinaciones):** El modelo tiene estrictamente prohibido generar información, funciones o soluciones que no existan dentro de los manuales proporcionados. Las alucinaciones en este contexto representan un riesgo crítico que degradará la experiencia del usuario.
- **Privacidad y Seguridad de la Información:** Por políticas de protección de datos, el sistema de IA no debe procesar, solicitar, ni almacenar datos sensibles de los clientes corporativos (tales como credenciales de acceso, tokens de API o saldos financieros) dentro del historial de chat o en la generación de prompts.
- **Trazabilidad de Respuestas:** Es un requerimiento imperativo que cada respuesta entregada por el agente referencia explícitamente la fuente de origen, indicando la página específica, la sección del manual o el enlace de la documentación de donde extrajo el contexto para formular la solución.

1.4 Respuestas a Consultas de Evaluación (Requerimientos del Docente)

1. Los manuales tienen muchos flujos e información detallada. ¿Cómo los procesarán? Para no romper la lógica de los manuales (como los flujos paso a paso o las listas de instrucciones), decidimos no cortar el texto simplemente por una cantidad rígida de caracteres, ya que eso dejaría instrucciones a medias. En su lugar, aplicamos **Fragmentación Semántica (*Recursive Character Text Splitting*)** usando LangChain. Esto nos permite dividir los documentos respetando su estructura natural (como párrafos y saltos de línea), asegurando que el modelo (LLM) reciba el contexto completo y no pierda el hilo conductual de los procesos detallados.

1.5 Evolución hacia un Agente Autónomo (Evaluación Parcial 2)

Para escalar la solución más allá de un sistema de búsqueda simple (RAG), en esta segunda fase se evolucionó la arquitectura hacia un Agente Inteligente Funcional. El objetivo de esta mejora es dotar al sistema de autonomía en la toma de decisiones, capacidad de utilizar herramientas específicas (como calculadoras de SLA o redacción de escalamientos) y procesos de memoria a corto plazo para mantener el contexto en flujos prolongados de soporte técnico.

2. ¿Costo y beneficio de la solución?

- **Beneficio:** El mayor impacto es el tiempo: pasamos de hacer esperar al cliente corporativo hasta 24 horas a entregarle una respuesta precisa en menos de 10 segundos. Al automatizar la resolución de al menos el 50% de los tickets operativos repetitivos, liberamos a los ingenieros de soporte para que puedan enfocarse en incidencias arquitectónicas o de código que sí requieren un análisis humano complejo.
- **Costo:** Logramos mantener los costos operativos prácticamente en cero. Diseñamos una arquitectura híbrida donde el modelo de *embeddings* (HuggingFace) y la base de datos vectorial (ChromaDB) se ejecutan de forma 100% local, eliminando el pago de suscripciones a bases de datos cloud. Además, la inferencia la maneja Llama 3.1 a través de la API gratuita de Groq, lo que nos otorga un rendimiento altísimo (usando LPUs) sin gastar presupuesto en tokens.

3. ¿Cómo se hará el flujo de ingesta? ¿Cuándo hay que actualizar la información?

Para resolver esto, la clave está en el uso de metadatos. Durante la fase de *chunking*, a cada fragmento de texto le inyectamos un identificador único del documento (`doc_id`). De esta forma, cuando SoftTech actualiza un manual, no tenemos que procesar toda la base de datos vectorial. El sistema simplemente busca y elimina de ChromaDB los vectores asociados al `doc_id` obsoleto y luego ingiere solo el nuevo PDF. En un entorno de producción, esto se automatiza conectando un evento (*webhook*) desde el repositorio de documentos de la empresa, para que dispare el script de actualización (`ingest.py`) de manera automática apenas detecte un cambio.

4. ¿Cómo se medirá si el sistema entrega buenas respuestas? Para asegurarnos de que el sistema sea confiable y no esté alucinando, nos basamos en dos métodos de validación:

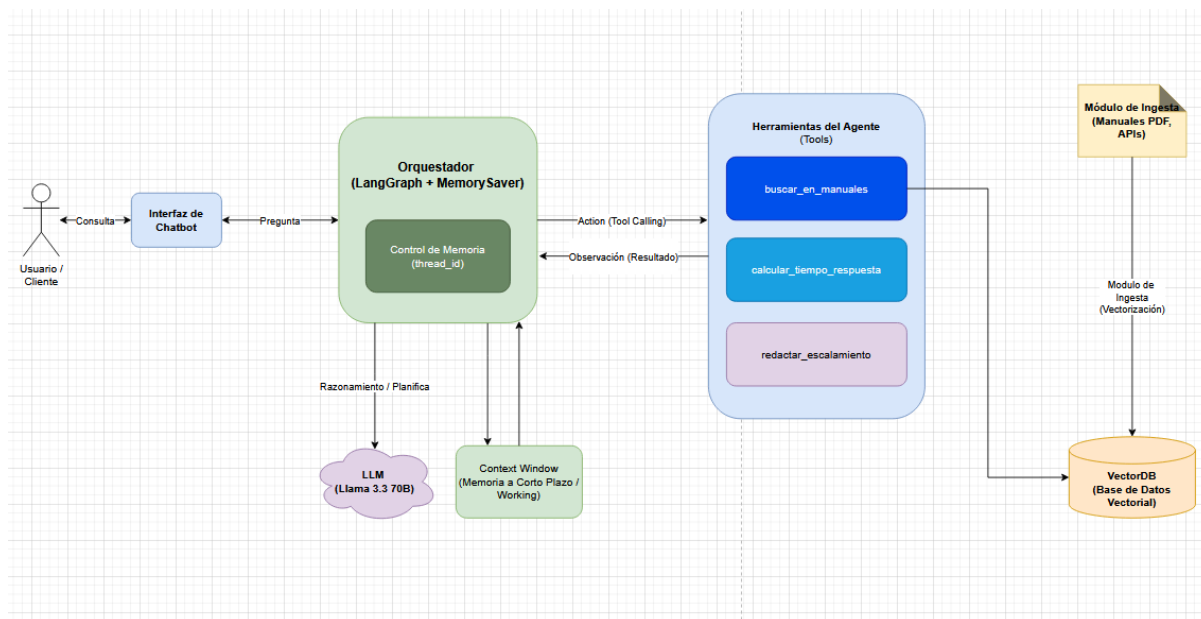
1. **Validación Técnica (Trazabilidad y *Grounding*):** A nivel de *backend*, la regla es estricta: el 100% de las respuestas deben incluir la cita exacta de origen (documento y página). Además, sometemos al sistema a pruebas de estrés con preguntas "fuera de contexto" para comprobar que el agente respete su barrera de seguridad y derive el caso a Nivel 2 en lugar de improvisar o adivinar una respuesta.
2. **Validación del Usuario (*Human-in-the-loop*):** En la interfaz del chat (Streamlit), incorporaremos un sistema de calificación simple (pulgar arriba/abajo) para que el propio cliente confirme si la respuesta resolvió su problema. Si la calificación es negativa, el ticket escala inmediatamente a un operador humano y el *log* de esa conversación se almacena para

que nosotros podamos refinar el *System Prompt* o ajustar la técnica de partición de los textos.

2. Arquitectura de la solución (Apartado D)

2.1 Diagrama de arquitectura del sistema.

A continuación, se presenta el esquema general de la arquitectura del sistema. Este diagrama ilustra el flujo de datos completo, desde la interacción inicial del usuario en el frontend, la recuperación semántica de la documentación corporativa en la VectorDB, hasta la generación de la respuesta final estructurada por el LLM a través del orquestador.



2.2 Descripción de componentes (Orquestador, VectorDB, LLM, Interfaz).

La arquitectura diseñada para SoftTech Solutions sigue un patrón RAG clásico, separando la recuperación de información de la generación de lenguaje. Cada componente ha sido seleccionado para garantizar baja latencia y alta precisión:

- **Interfaz de Usuario (Frontend):** Corresponde al canal de entrada, integrado como un widget de chat en la plataforma de soporte de SoftTech Solutions. Captura la consulta en lenguaje natural del cliente corporativo y muestra la respuesta final junto con las citas y enlaces de referencia.
- **Orquestador (LangChain):** Evolucionó del enrutamiento lineal de LangChain a un esquema basado en grafos (LangGraph). Actúa como el "cerebro" aplicando el patrón *Plan-and-Execute*, evaluando la consulta del usuario, decidiendo qué herramienta utilizar y manteniendo el estado de la memoria (`thread_id`).

- **Base de Datos Vectorial (VectorDB):** (Se utilizará una solución como *ChromaDB* o *Pinecone*). Este componente almacena la documentación de la empresa (manuales PDF, guías de API e historial de tickets) previamente transformada en representaciones numéricas (*embeddings*). Su función es realizar una búsqueda semántica ultrarrápida para recuperar únicamente los fragmentos de texto (*chunks*) más relevantes que contienen la solución al problema del usuario.
- **Modelo de Lenguaje (LLM):** Se actualizó a Llama 3.3 70B Versatile (vía Groq). Se justificó este cambio debido a que los modelos de mayor parametrización son sustancialmente superiores en la ejecución de tareas de *Tool Calling* (uso de herramientas) y razonamiento lógico, evitando errores de sintaxis en la orquestación.
- **Módulo de Ingesta y Embeddings:** Componente asíncrono o *backend* encargado de preprocesar los documentos crudos de SoftTech Solutions. Limpia el texto, lo divide en fragmentos lógicos (*Chunking*) y lo convierte en vectores multidimensionales mediante un modelo de embeddings (ej. *text-embedding-3-small*) para su almacenamiento en la VectorDB.

3. Diseño e implementación de pipeline RAG (Apartado C)

3.1 Fuentes de datos internas (Manuales PDF, API, Historial de tickets).

El pipeline de recuperación se alimenta exclusivamente del conocimiento interno de SoftTech Solutions, abarcando tanto datos no estructurados como estructurados:

- **Manuales de Usuario (PDF):** Documentación extensa que detalla el uso operativo y los flujos de configuración del SaaS.
- **Documentación de API (Markdown/HTML):** Archivos técnicos con los *endpoints* y guías para desarrolladores.
- **Historial de Tickets (CSV/JSON):** Un repositorio tabular con las consultas recurrentes ya resueltas, optimizando la respuesta a problemas comunes.

3.2 Flujo de ingesta, fragmentación (Chunking) y vectorización.

Para abordar la complejidad de los manuales que contienen flujos e instrucciones detalladas paso a paso, el sistema no utilizará un corte de texto rígido por cantidad de caracteres. En su lugar, se implementará una técnica de **Fragmentación Semántica (Recursive Character Text Splitting)** mediante LangChain. Esta estrategia asegura que los párrafos, listas o flujos lógicos no se dividan por la mitad, preservando el contexto original de la instrucción. Una vez divididos, los fragmentos (*chunks*) son procesados por un modelo de embeddings (ej. *text-embedding-3-small*) que transforma el texto en vectores multidimensionales. Para gestionar la actualización constante de los manuales, el flujo de ingesta incluye el uso de **metadatos**. Cuando SoftTech actualiza un documento, el sistema identifica el identificador único del archivo (*doc_id*), elimina los vectores obsoletos de la VectorDB e ingesta los nuevos fragmentos, garantizando que la IA siempre responda con la versión más reciente sin necesidad de procesar toda la base de datos.

3.3 Mecanismos de recuperación de datos.

Cuando el cliente ingresa una consulta, el orquestador vectoriza la pregunta y realiza una **Búsqueda de Similitud Semántica (Similitud del Coseno)** dentro de la VectorDB. El sistema identifica matemáticamente los fragmentos de conocimiento más cercanos a la intención del usuario. Para maximizar la precisión de la recuperación, se aplica un esquema de búsqueda híbrida apoyado en el filtrado de metadatos. El orquestador selecciona únicamente los *Top-K* documentos más relevantes (por ejemplo, los 3 fragmentos con mayor puntuación) y los inyecta en el contexto del modelo de

lenguaje, descartando el resto de la información para no saturar la ventana de contexto del LLM y evitar fugas de información.

4. Formulación de prompts (Apartado B)

4.1 Diseño del System Prompt del agente.

Para garantizar que el modelo de lenguaje opere bajo los estrictos parámetros de seguridad de SoftTech Solutions, se ha diseñado el siguiente *System Prompt* principal utilizando técnicas de delimitación de contexto (*Context Boundaries*) y asignación de rol:

System Prompt: "Eres un asistente de soporte técnico avanzado de Nivel 1 para el software SaaS de SoftTech Solutions. Tu objetivo es resolver problemas de clientes corporativos razonando paso a paso. TIENES ACCESO A HERRAMIENTAS. Antes de responder, PIENSA:

1. ¿La pregunta es sobre cómo hacer algo, manuales o errores? Usa la herramienta 'buscar_en_manuales'.
2. ¿El usuario pregunta por tiempos de espera o SLAs? Usa la herramienta 'calcular_tiempo_respuesta'.
3. ¿Es un problema que no puedes resolver? Usa la herramienta 'redactar_escalamiento'. Si el usuario reporta un problema muy general, primero pídele más detalles específicos ANTES de usar cualquier herramienta."

Regla crítica: Debes responder a la consulta del usuario basándote única y exclusivamente en los fragmentos de contexto proporcionados a continuación. Tiene estrictamente prohibido utilizar conocimiento externo, inventar funciones o asumir soluciones que no estén explícitamente en el texto.

INSTRUCCIONES:

1. Analiza la pregunta del usuario y busca la respuesta en el CONTEXTO RECUPERADO.
2. Si la respuesta se encuentra en el contexto, redacta una solución paso a paso.
3. Al final de tu respuesta, debes incluir obligatoriamente la referencia exacta de dónde obtuviste la información (ejemplo: 'Fuente: Manual de Usuario, pág. 12').

4. Si el contexto proporcionado no contiene la información necesaria para responder a la pregunta, **NO INVENTES NINGUNA RESPUESTA**. Simplemente responde: 'Lamento no poder ayudarle con esto. No encuentro la información exacta en nuestra documentación actual. Transferiré su caso a un ejecutivo de Nivel 2.'

4.2 Justificación de la estructura para evitar alucinaciones y mantener el rol.

La estructura del prompt anterior fue diseñada estratégicamente para cumplir con los requerimientos técnicos y de negocio del proyecto:

- **Definición y Enjaulamiento del Rol:** Al iniciar declarando que es un "asistente de soporte técnico experto de Nivel 1", se calibra el tono del LLM para que sea profesional y corporativo.
- **Mitigación Activa de Alucinaciones (Contextual Grounding):** La inclusión de la regla crítica en mayúsculas y la instrucción específica del paso 4 ("NO INVENTES NINGUNA RESPUESTA") actúa como un cortafuegos. Esto bloquea la tendencia probabilística del LLM a "adivinar" e inventar funciones que el software no posee, mitigando un riesgo crítico para la experiencia del usuario.
- **Trazabilidad y Transparencia:** La instrucción del paso 3 fuerza al modelo a extraer y mostrar los metadatos de los documentos recuperados. Esto cumple con la necesidad imperativa de que cada respuesta referencie explícitamente la fuente original (página o sección), permitiendo a los clientes o supervisores auditar la veracidad de la información entregada por el agente.
- **Esquemas de Planificación de Tareas:** El prompt fue rediseñado bajo el esquema *Plan-and-Execute*. Antes de generar texto, el agente es forzado a secuenciar sus actividades según prioridades (evaluar intención -> seleccionar herramienta -> ejecutar -> responder).

5. Integración de Herramientas y Memoria

5.1 Configuración de Herramientas Autónomas (Tools)

Para dotar al agente de autonomía frente a las condiciones del entorno, se configuraron tres herramientas específicas que el modelo puede invocar a discreción:

- **buscar_en_manuales (Consulta):** Conecta el agente con el pipeline RAG de ChromaDB para extraer conocimiento técnico.

- **calcular_tiempo_respuesta (Razonamiento):** Función lógica que evalúa cadenas de texto ("alta", "media", "baja") para calcular Acuerdos de Nivel de Servicio (SLA) sin consultar la base de datos.
- **redactar_escalamiento (Escritura):** Herramienta de formato que estructura un ticket formal cuando el agente determina que la complejidad supera el Nivel 1.

5.2 Implementación de Memoria de Contenido

Para asegurar la continuidad en flujos prolongados de soporte, se integró el framework LangGraph utilizando el módulo **MemorySaver**. A diferencia de los modelos *stateless* (sin memoria), el sistema ahora genera un identificador único de sesión (`thread_id`) mediante la librería **uuid** de Python. Esto permite inyectar el historial de mensajes de forma dinámica en la ventana de contexto del LLM, logrando que el agente recuerde el nombre del usuario y los problemas descritos en interacciones previas dentro de la misma sesión web.

6. Evidencia de Pruebas y Toma de Decisiones (IE6 e IE10)

6.1 Evidencia de Razonamiento y Toma de Decisiones

Para demostrar que el comportamiento del agente responde a las condiciones del entorno automatizado y no es una simple cadena de texto, se ejecutó el sistema en modo consola (verbose=True).

```
(venv) D:\Inteligencia artificial\python -m src.agent  
D:\Inteligencia artificial> python -m site-packages\langgraph\checkpoint\serde\encrypted.py:5: LangChainPendingDeprecationWarning: The default value of 'allowed_objects' will change in a future version. Pass an explicit value (e.g., allowed_objects='messages' or allowed_objects='core') to suppress this warning.  
    from langgraph.checkpoint.serde.jsonplus import JsonPlusSerializer  
Cargando modelo de embeddings local (HuggingFace)...  
WARNING: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.  
Loading weights: 100%|███████████████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 30492.19it/s]  

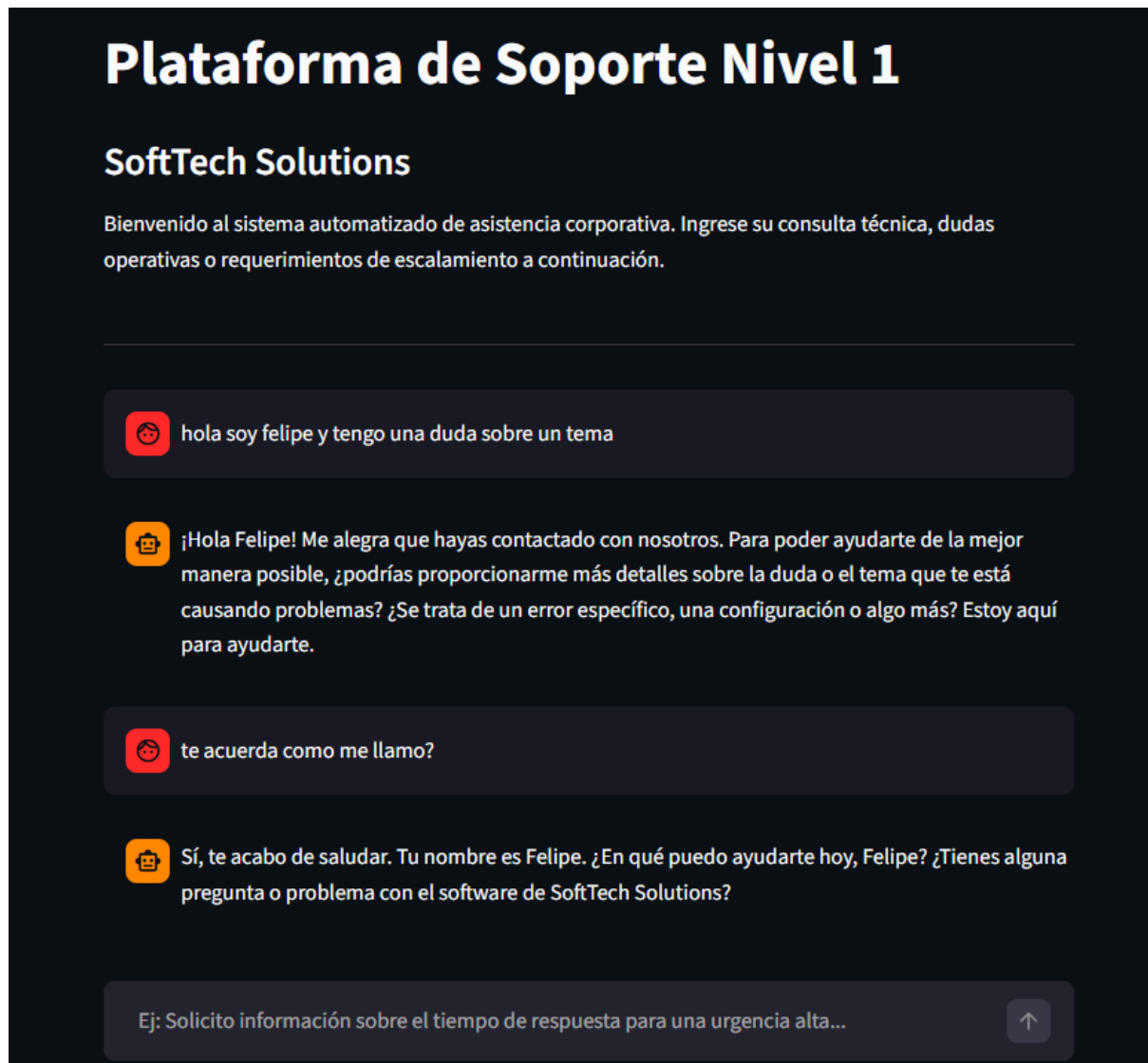

|                         | Status     | Details |
|-------------------------|------------|---------|
| embeddings.position_ids | UNEXPECTED |         |

  
Notes:  
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.  
Cargando Base de Datos Vectorial desde: vector_store...  
Iniciando Agente Funcional (LangGraph) con Memoria...  
  
Usuario: hola soy felipe y tengo una duda sobre un tema  
D:\Inteligencia artificial>src\agent.py:41: DeprecationWarning: create_react_agent has been moved to 'langchain.agents'. Please update your import to 'from langchain.agents import create_agent'  
. deprecated in LangGraph V1.0 to be removed in V2.0.  
    _agent_executor = create_react_agent(  
  
Agente: ¡Hola Felipe! Me alegra que hayas contactado con nosotros. Para poder ayudarte de la mejor manera posible, ¿podrías proporcionarme más detalles sobre la duda que tienes? ¿Se refiere a un error específico, una configuración o algo más? Esto me permitirá entender mejor tu situación y ofrecerte una respuesta más precisa.  
  
-----  
Usuario: ¿Recuerdas mi nombre? y también ¿qué es el modelo lambda?  
  
Agente: ¡Hola de nuevo Felipe! Me alegra recordarte mi nombre. En cuanto a tu pregunta sobre el modelo lambda, según nuestros manuales oficiales, el modelo lambda se refiere a una arquitectura de referencia que se divide en tres capas: capa de lote (batch layer), capa de velocidad (speed layer) y capa de servir (serving layer). Esta arquitectura es utilizada para procesar y analizar grandes cantidades de datos de manera eficiente, y es especialmente útil para aplicaciones de machine learning y personalización. El modelo lambda es capaz de manejar tanto datos en lote como datos en tiempo real, lo que lo hace ideal para aplicaciones que requieren una respuesta rápida y precisa. ¿Te gustaría saber más sobre cómo se aplica el modelo lambda en nuestros sistemas?
```

Análisis de la evidencia: Como se observa en el log del sistema, el agente evalúa la solicitud del usuario, deduce qué herramienta necesita y la ejecuta de forma autónoma antes de formular la respuesta final.

6.2 Evidencia de Memoria a Corto Plazo en la Interfaz

Se sometió al sistema a una prueba de continuidad conversacional mediante la interfaz de Streamlit.



Análisis de la evidencia: La prueba confirma el correcto funcionamiento del módulo MemorySaver, demostrando que el orquestador retiene variables específicas (como el nombre del cliente y el contexto del problema) a lo largo de interacciones secuenciales.

7. Conclusiones y Reflexiones Individuales

7.1 Conclusión General del Proyecto

La implementación de la arquitectura RAG en SoftTech Solutions demostró ser una solución técnica altamente efectiva para descongestionar el soporte técnico de Nivel 1. Al integrar un modelo de embeddings de ejecución local y un LLM de baja latencia bajo un entorno restringido, se logró automatizar la respuesta a consultas repetitivas con tiempos de resolución inferiores a 10 segundos. Además, el uso estricto de la documentación oficial como única fuente de verdad garantizó la precisión de la información entregada, permitiendo escalar el servicio de soporte sin incurrir en altos costos de entrenamiento y sin comprometer la privacidad de los datos corporativos de los clientes.

7.2 Reflexión Personal - Felipe Monsalve

Al armar este proyecto me encontré con varios problemas que me sirvieron para aprender. Lo primero fue hacer una buena arquitectura en el entorno virtual; me costó dejarlo funcionando bien, pero me quedó claro que si no trabajas de forma ordenada desde el principio, después el código es un desastre.

Un cambio importante fue dejar de lado la API de OpenAI para usar HuggingFace y Groq. Fue una buena decisión porque, además de ahorrar plata, me di cuenta de que se pueden hacer cosas potentes sin depender de empresas externas.

Lo más difícil fue el tema de las alucinaciones de la IA. En una empresa no puedes dejar que un bot invente cualquier cosa porque quedas mal con los clientes. Por eso, lo que más me costó pero logré hacerlo fue "enjaular" al modelo para que sea confiable y no responda tonteras.

7.3 Reflexión Personal - Gabriel Berman

Para mí, este proyecto dejó claro lo importante que es trabajar en equipo usando Git y GitHub. Cuando tuvimos problemas al mezclar los cambios del repositorio y aprendimos a resolverlos, se notó lo necesario que es eso en un contexto real. También nos permitió ir integrando el trabajo sin desordenarnos y entender mejor cómo coordinar el código entre los dos.

Por otro lado, los prompts me hicieron cambiar de vista la forma en que veía la IA. Me di cuenta de que no se trata solo de usarla, sino de saber como pedirle las cosas, con instrucciones claras, para que entregue justo lo que uno necesita. Mientras más preciso eres, mejor funciona.

Al final, esto me hizo pensar en el soporte técnico a futuro. Herramientas como la que hicimos no vienen a reemplazar a las personas, sino a ayudar con tareas repetitivas. Así, el trabajo se enfoca más en analizar problemas y tomar decisiones, donde el criterio humano sigue siendo importante.

8. Referencias Bibliográficas

Chroma. (2024). *ChromaDB Documentation: The AI-native open-source embedding database.* Recuperado de <https://docs.trychroma.com/>

Groq. (2024). *Groq Cloud Documentation: Llama 3.1 Inference on LPU Architecture.* Recuperado de <https://console.groq.com/docs>

Hugging Face. (2024). *Sentence-Transformers: all-MiniLM-L6-v2 model.* Recuperado de <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

LangChain. (2024). *LangChain Documentation: Retrieval-Augmented Generation (RAG).* Recuperado de https://python.langchain.com/docs/use_cases/question_answering/

Streamlit. (2024). *Streamlit API Reference: Building Chat Interfaces.* Recuperado de <https://docs.streamlit.io/>

LangChain. (2024). *LangGraph Documentation: Building Stateful Multi-Actor Applications.* Recuperado de <https://langchain-ai.github.io/langgraph/>